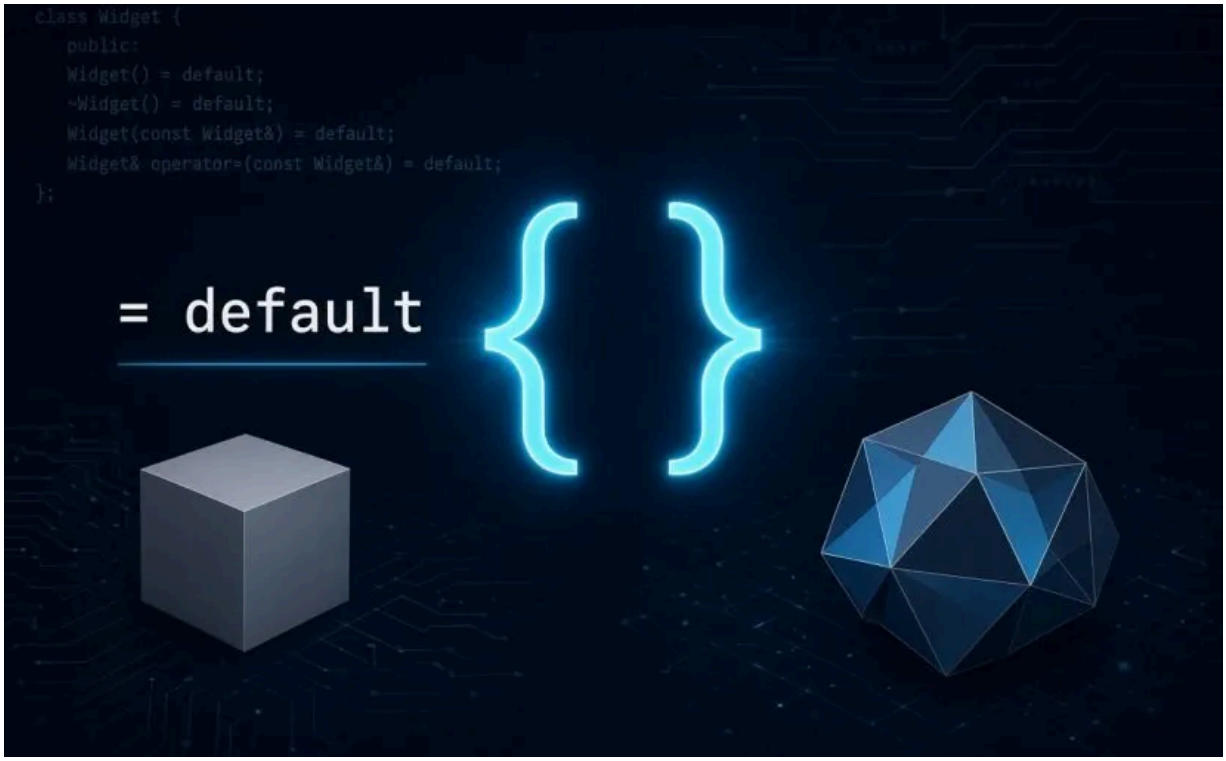
Daniel Lemire's blog

Daniel Lemire is a software performance expert. He ranks among the top 2% of scientists globally (Stanford/Elsevier 2025) and is one of GitHub's top 1000 most followed developers.



In C++, = default and {} are not the same constructor

2 November 2026 · 3 min

You can write an empty default constructor as `span() = default` or as `span() {}`. It can have remarkable effects on performance.

```
struct span {
  const char *p;
  int len;
  const char *end;
  span() = default; // or span() {}
  span(const char *s, int n) : p(s), len(n), end(s + n) {}
};
```

The constructor `span() = default` on its first declaration makes `span` trivial. The constructor `span() {}` is user-provided, so `span` is not trivial.

Constructor	Declaration	What happens	Members
`span() = default`	`span s;`	nothing	uninitialized
`span() = default`	`span s{}`	the whole object is zeroed, padding included; the constructor does not run	zero
`span() {}`	`span s;`	the empty constructor is called; the compiler can delete the call	uninitialized
`span() {}`	`span s{}`	the empty constructor is called; the object is not zeroed	uninitialized

An example

This `span` is a start pointer, a length, and an end pointer. A `box` holds one span. The expression `box()` value-initializes that span. The loop assigns a value-initialized box when `i` reaches `n`, and otherwise a span of length `i + 1`:

```
struct box {
  span s;
  box() : s() {}
  box(span in) : s(in) {}
};
void consume(box &);
void walk(const char *s, int n) {
  for (int i = 0;; i++) {
    box b;
    b = i == n ? box() : box(span(s, i + 1));
    if (i == n) { return; }
    consume(b);
  }
}
```

The `box()` path runs once, as the loop exits, and that span is not passed to `consume`.

With `span() {}`, GCC 16 (`-O2 -march=x86-64`) writes the three fields directly where `consume` expects them:

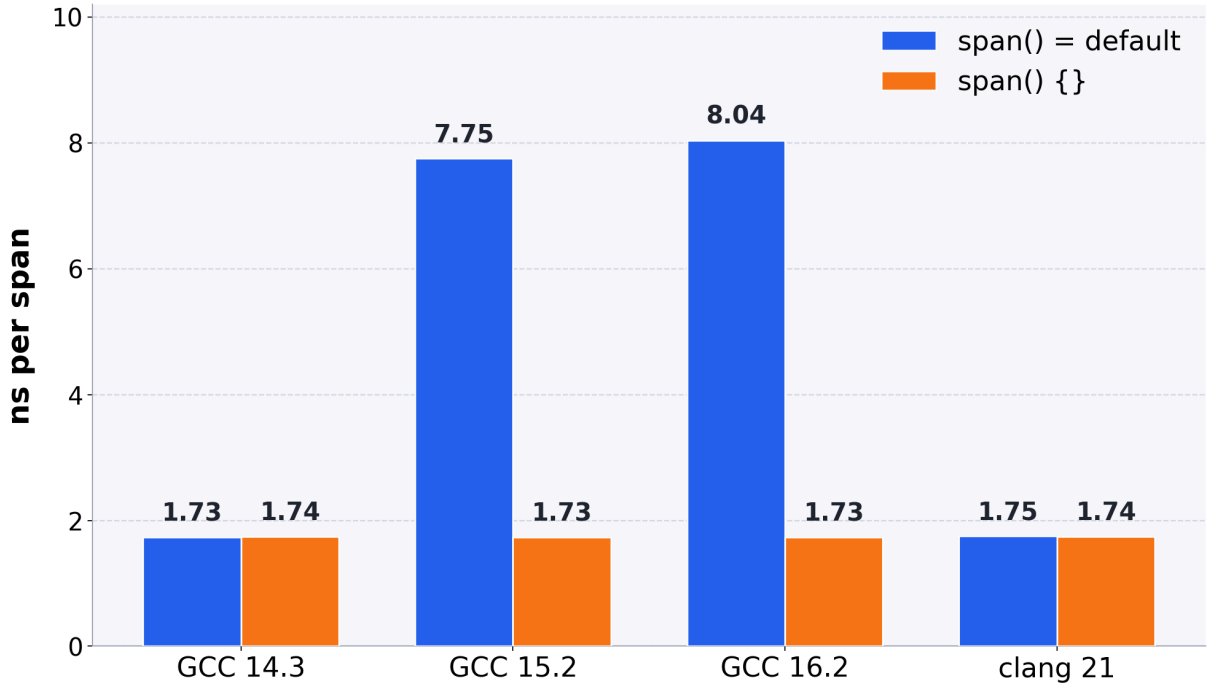
```
.L4:
    mov eax, ebx
    mov QWORD PTR [rsp+16], rbx
    mov rdi, rsp
    add rbx, 1
    sub eax, ebp
    mov QWORD PTR [rsp], rbp
    mov DWORD PTR [rsp+8], eax
    call    consume(box&)
    cmp rbx, r12
    jne .L4
```

With `span() = default`, GCC 16 stores the end pointer, `rbx`, straight into the box. The start pointer and the length go through a temporary, and the first 16 bytes of that temporary are copied back.

```
.L3:
    mov eax, ebx
    mov QWORD PTR [rsp+32], rbp
    mov rdi, rsp
    sub eax, ebp
    mov QWORD PTR [rsp+16], rbx
    add rbx, 1
    mov DWORD PTR [rsp+40], eax
    movdqa  xmm0, XMMWORD PTR [rsp+32]
    movaps  XMMWORD PTR [rsp], xmm0
    call    consume(box&)
    cmp rbx, r12
    jne .L3
```

I time the best of five runs of 100 million spans, on one core of an Intel Xeon Gold 6548N (Emerald Rapids), with `-O2 -march=x86-64`. My code is available.

compiler	`span() = default`	`span() {}`
GCC 14.3	1.73 ns/span	1.74 ns/span
GCC 15.2	7.75 ns/span	1.73 ns/span
GCC 16.2	8.04 ns/span	1.73 ns/span
clang 21	1.75 ns/span	1.74 ns/span



With GCC 15 and GCC 16, the `= default` version is over four times slower.

Daniel Lemire, "In C++, = default and {} are not the same constructor," in *Daniel Lemire's blog*, November 2, 2026, https://lemire.me/blog/2026/11/02/default-and-braces-are-not-the-same-constructor/. [BibTeX]

PUBLISHED BY

Daniel Lemire
A computer science professor at the University of Quebec (TELUQ).
View all posts by Daniel Lemire →

November 2, 2026 Daniel Lemire

You can also subscribe by email to this blog (non-commercial, no ads, weekly email).

How to post code (C, C++, Java, Python, etc.):

Wrap your code in backticks, like this: